LAYOUT COMMAND CONTROL DIY NODE DEVELOPMENT

BY BOB GAMBLE, MMR® #800

AGENDA

- Disclaimer
- LCC Background
- Objectives
- Resources
- Hardware
- Software Library
- Software Application
- My Experience
- Recommendations

DISCLAIMER

- By training I am a Mechanical Engineer, not Electrical or Programming. I did have a career in computer and software development, so I did pick up a few things along the way. Osmosis can be wonderful.
- Self-taught in programming though I have had a course or two in college and professionally.
- I muddle through on circuit design and as Ringo said “With a little help from my friends.”

LCC BACKGROUND

- Had produced a number of LocoNet nodes using Arduino Nanos and Megas but wanted to upgrade to LCC. Bought a starter kit in 2020.
- I started deploying LCC on my layout in 2021 with Tortoise controls using RRCirKits products. Worked with Dick Bronson to use sampling mode on I/O lines and wrote an article in the NMRA magazine (Jan 2023) on the topic.
- Investigated an ESP32 based LCC development board by Mike Dunston but decided the development environment was too complex for me.
- Deployed block occupancy (2022) and signaling (2023)
- Started using Raspberry Pi Pico and decided to start on the LCC DIY path in 2024.
- Layout has 10 LCC nodes: 7 Towers w/9 SMD-8 & 5 BOD-8, 2 Signal 32H, 1 DIY Pico for lighting
  One LocoNet node for Turntable & Roundhouse control to be replaced with DIY LCC

MY LAYOUT – SOUTHERN PIEDMONT

- **RR Prototype** — Southern Railway
- **Scale** — HO
- **Era** – Early 1960s, Diesel
- **Scenery complete** – 90%
- **Configuration** — Overlapping folded dog bone
- **Layout Size** — 25 by 19 ft
- **Location in Home** —Third floor attic
- **Approximate Square Feet** – 450 plus
- **DCC System** — Digikeijs/YAMORC w/ WiThrottle or Engine Driver
- **Operation** — Fictional branches of the Southern, with main yard, interchanges, local industries.
- **Ops-Method –** JMRI manifests/switch lists
- **Dispatching** – JMRI Dispatcher Panel.
- **Max Operators** – 5, yardmasters & road crews

OBJECTIVES

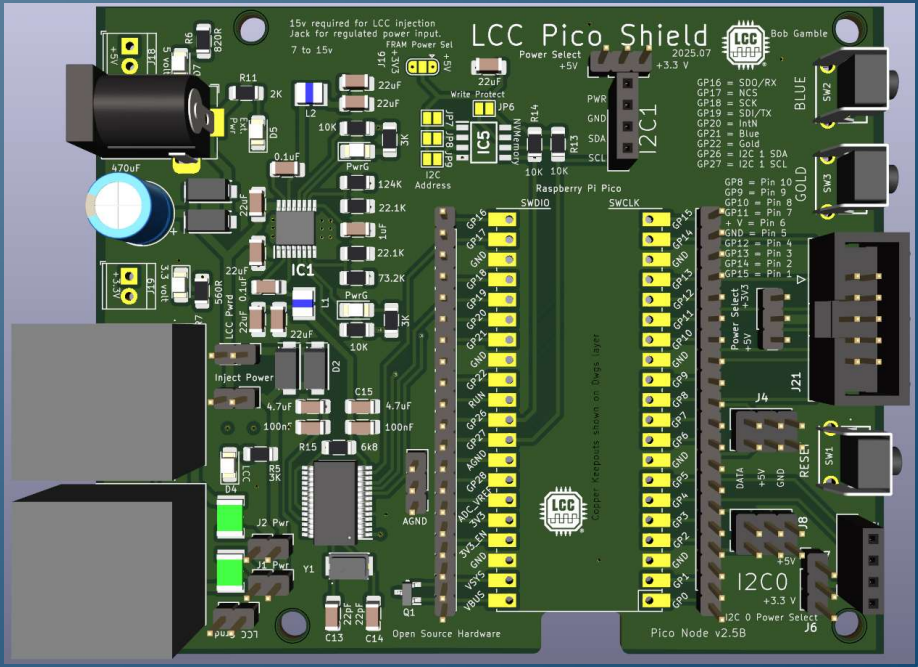
- Ease of maintenance using the LCC Configuration tools in JMRI
- Cross-node communication (events)
- Familiar development environment (Arduino IDE)
- Goal: to replace my DIY LocoNet turntable & roundhouse controller with one based on LCC.

RESOURCES

- Friends Mike Dunston and David Harris provided advice and guidance on hardware and software.
- LCC community groups for questions and answers both mine and from others.
- Github for example software.
- Videos, blogs, and books.

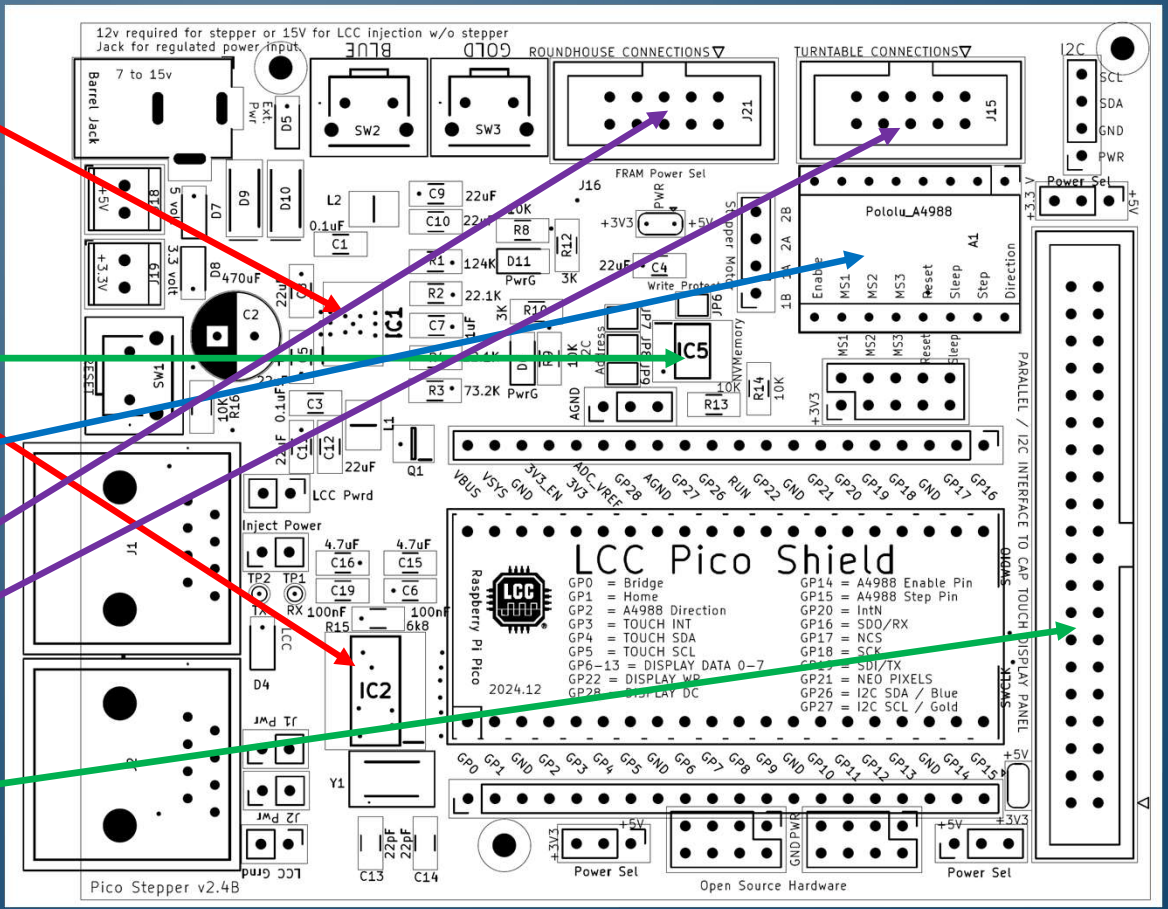
HARDWARE

- I spent a lot of time reading component datasheets on Mouser and searching for example reference designs and schematics.
- The LCC Pico node was designed with the help of my friends. Thanks Ringo.
- Designed in KiCad with PCB from China
- Dual power regulator provides 3.3v and 5v from either LCC cable or external source. If powered externally the node may be configured to inject power to the LCC cable.
- CAN transceiver and NVM support as the RPi Pico lacks these features



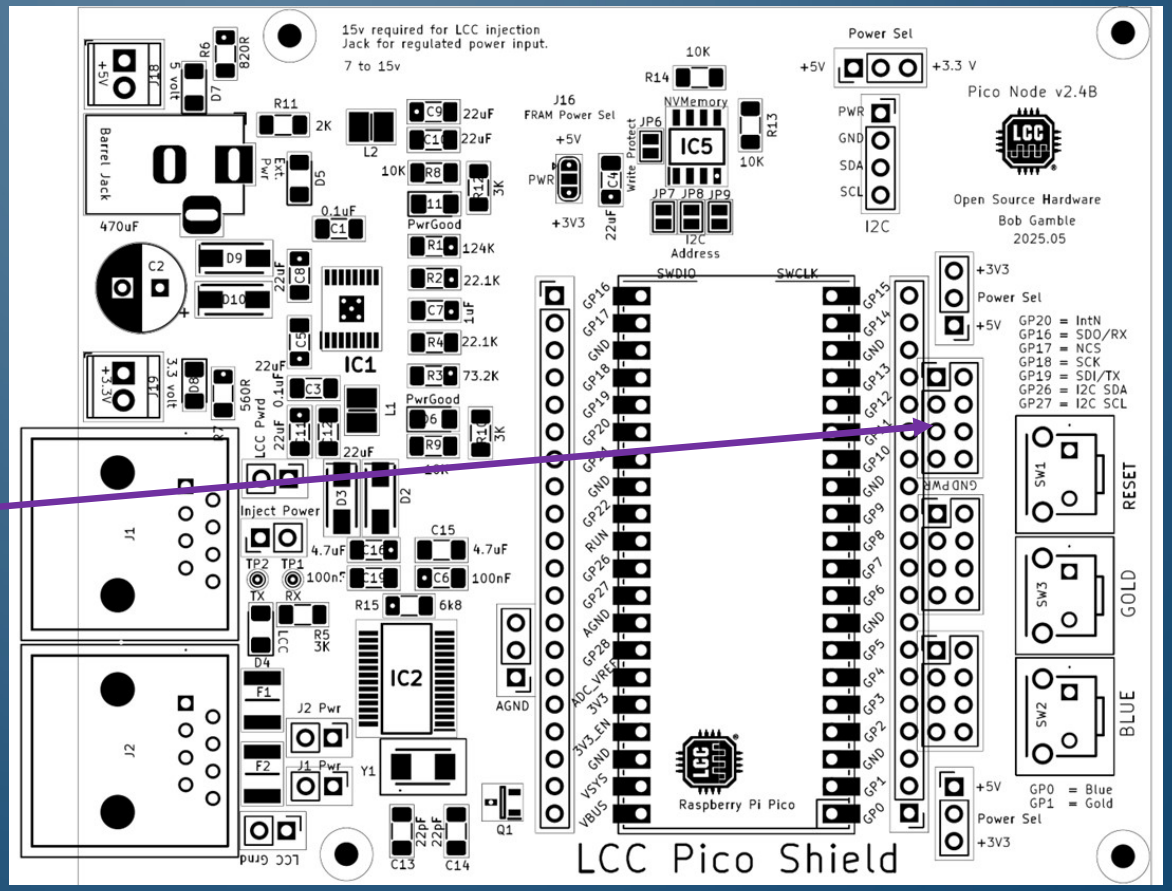
BOARD ONE: TURNTABLE

- Dual Power controller
  - 3.3 volts
  - 5 volts
- CAN transceiver
- NVM on I2C
- Stepper driver socket
  - External power
- Appl specific connectors
  - Roundhouse
  - Turntable
  - Touch Display



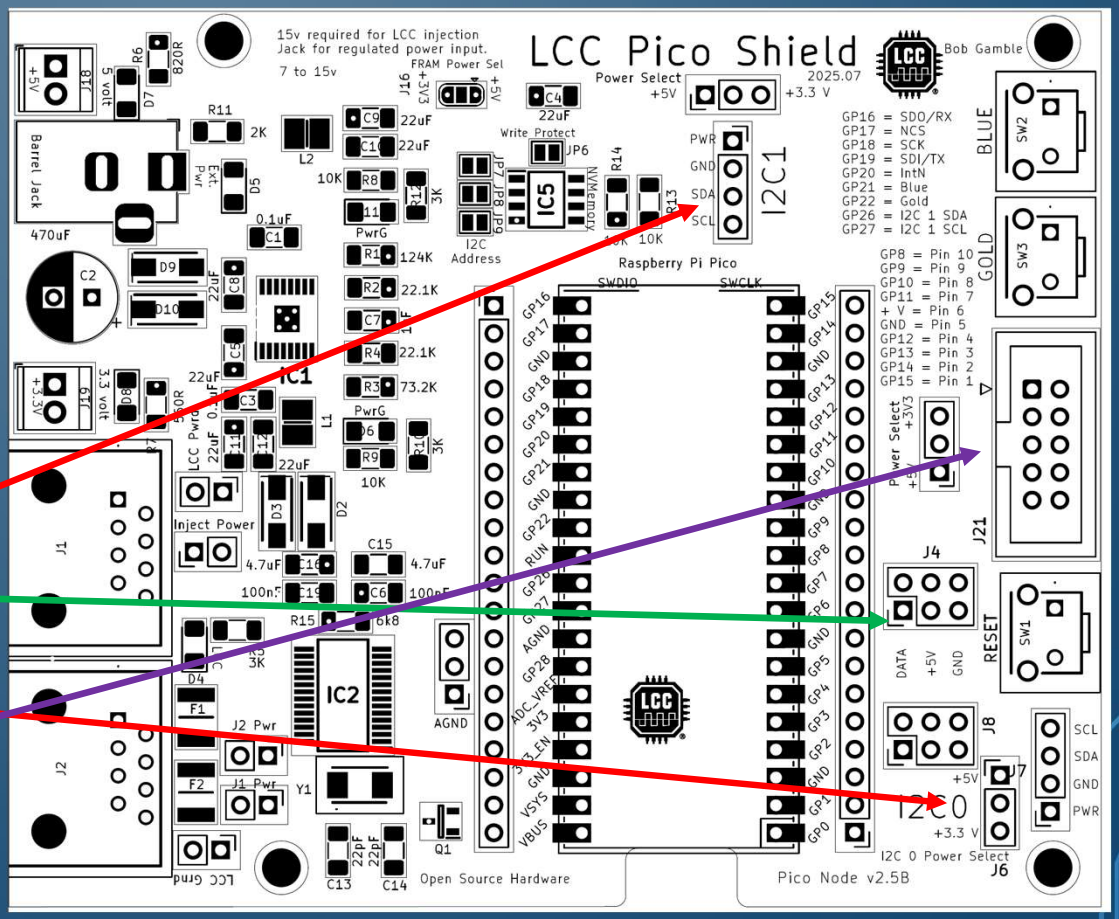
BOARD TWO: SIMPLIFIED DEVELOPMENT

- Dual Power controller
  - 3.3 volts
  - 5 volts
- CAN transceiver
- NVM on I2C
- Pico pin access



BOARD THREE: GENERAL USAGE

- Dual Power controller
  - 3.3 volts
  - 5 volts
- CAN transceiver
- NVM on I2C
- Appl specific connectors
  - NeoPixel (4)
  - I2C (2)
  - 10 Pin IDC (8 GPIO pins)

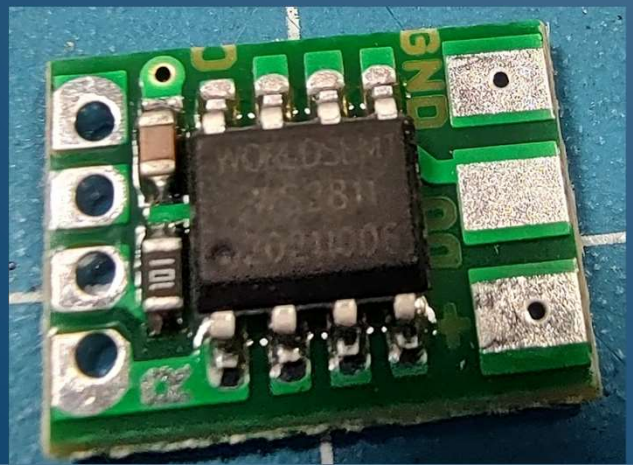


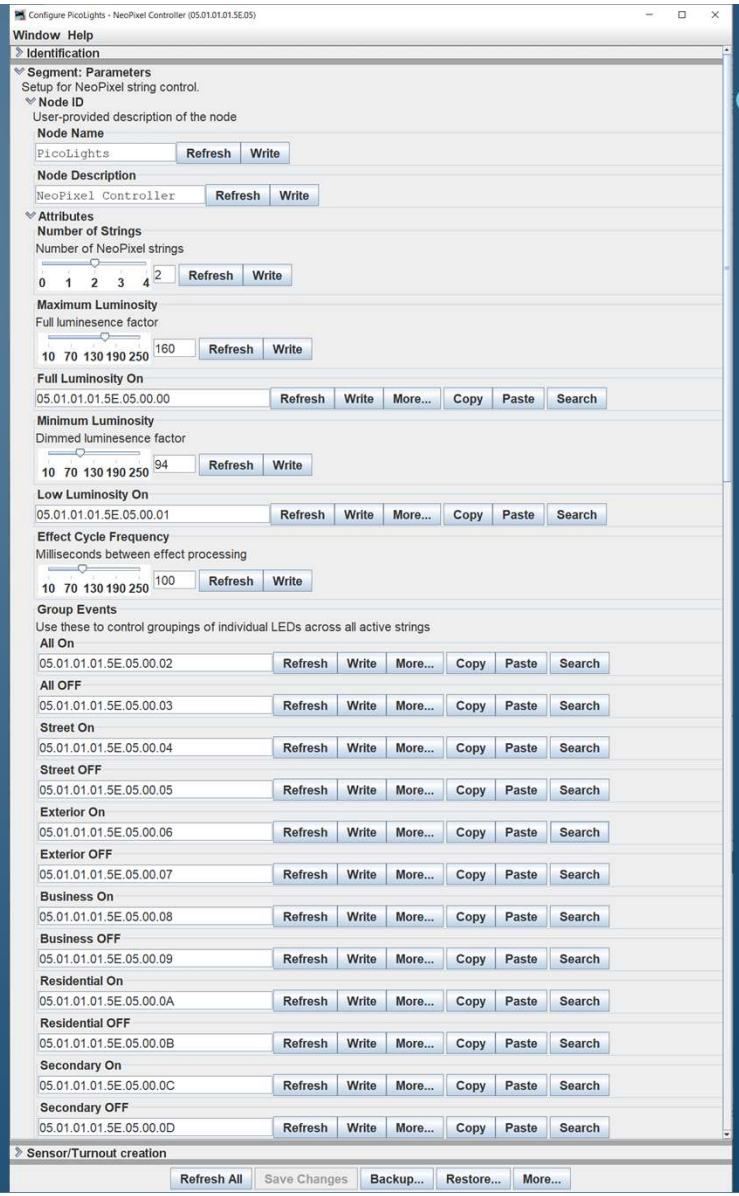
SOFTWARE LIBRARY

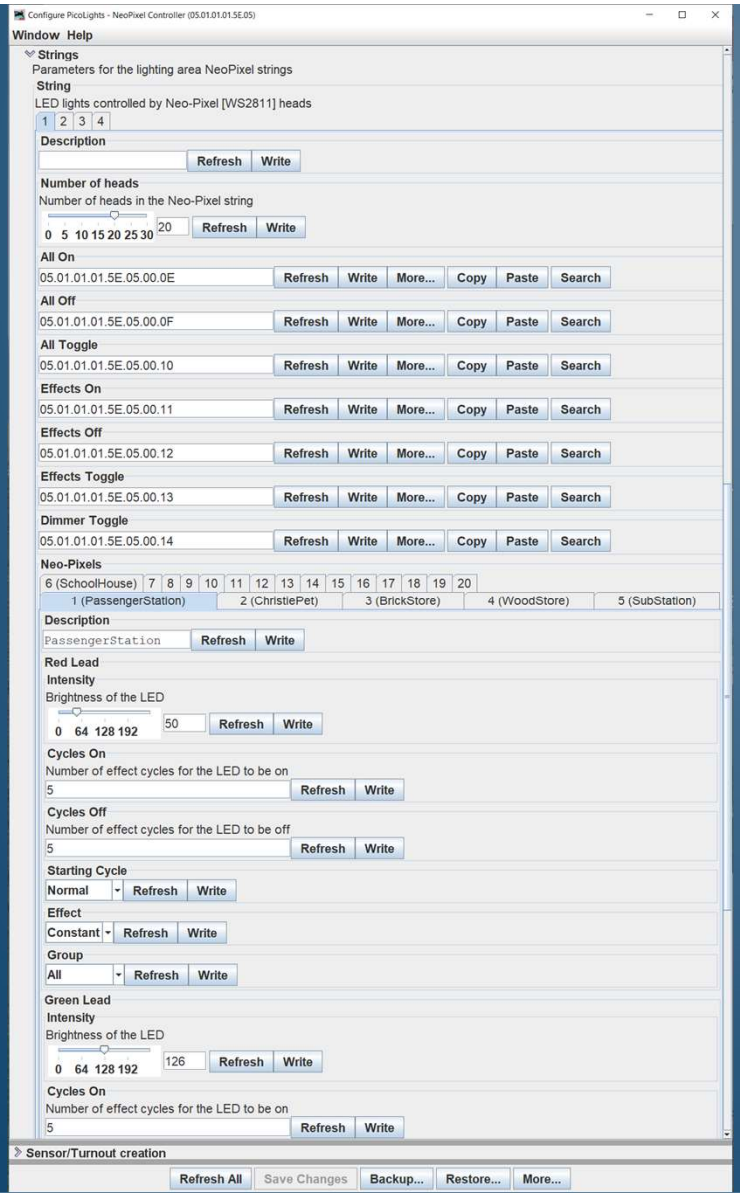
- Using the OpenLCB_Single_Thread library maintained by David Harris
  - Supported by the Arduino IDE
- Raspberry Pi Pico Arduino core, Arduino-Pico, by Earle Philhower, III.
- Various other libraries for specific functions
  - NeoPixel lights
  - Non-Volatile Memory
  - Servos
  - Stepper Motors
  - Displays

SOFTWARE APPLICATION

- Start small. Single function to drive a string of NeoPixel lights.
- Declarations to set up variables
- CDI and NVM Memory Structure
- Event handlers
- Main loop processing






Configure PicoLights - NeoPixel Controller (05.01.01.01.5E.05)
Window Help
Strings
Parameters for the lighting area NeoPixel strings
String
LED lights controlled by Neo-Pixel [WS2811] heads
1 2 3 4
Description
Refresh Write
Number of heads
Number of heads in the Neo-Pixel string
0 5 10 15 20 25 30
20
Refresh Write
All On
05.01.01.01.5E.05.00.0E
Refresh Write More... Copy Paste Search
All Off
05.01.01.01.5E.05.00.0F
Refresh Write More... Copy Paste Search
All Toggle
05.01.01.01.5E.05.00.10
Refresh Write More... Copy Paste Search
Effects On
05.01.01.01.5E.05.00.11
Refresh Write More... Copy Paste Search
Effects Off
05.01.01.01.5E.05.00.12
Refresh Write More... Copy Paste Search
Effects Toggle
05.01.01.01.5E.05.00.13
Refresh Write More... Copy Paste Search
Dimmer Toggle
05.01.01.01.5E.05.00.14
Refresh Write More... Copy Paste Search
Neo-Pixels
6 (SchoolHouse) 7 8 9 10 11 12 13 14 15 16 17 18 19 20
1 (PassengerStation) 2 (ChristiePet) 3 (BrickStore) 4 (WoodStore) 5 (SubStation)
Description
PassengerStation
Refresh Write
Red Lead
Intensity
Brightness of the LED
0 64 128 192
50
Refresh Write
Cycles On
Number of effect cycles for the LED to be on
5
Refresh Write
Cycles Off
Number of effect cycles for the LED to be off
5
Refresh Write
Starting Cycle
Normal
Refresh Write
Effect
Constant
Refresh Write
Group
All
Refresh Write
Green Lead
Intensity
Brightness of the LED
0 64 128 192
126
Refresh Write
Cycles On
Number of effect cycles for the LED to be on
5
Refresh Write
Sensor/Turnout creation
Refresh All Save Changes Backup... Restore... More...

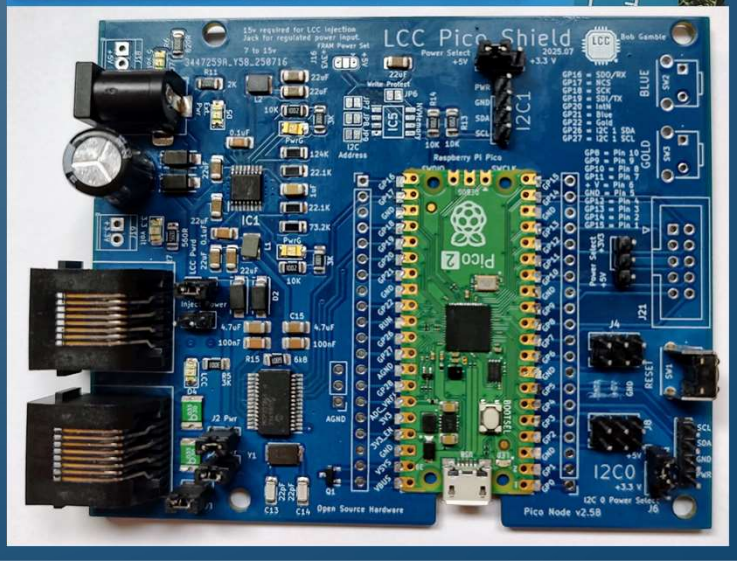
Configure PicoLights - NeoPixel Controller (05.01.01.01.5E.05)
Window Help
Neo-Pixels
6 (SchoolHouse) 7 8 9 10 11 12 13 14 15 16 17 18 19 20
1 (PassengerStation) 2 (ChristiePet) 3 (BrickStore) 4 (WoodStore) 5 (SubStation)
Description
PassengerStation
Refresh Write
Red Lead
Intensity
Brightness of the LED
0 64 128 192
50
Refresh Write
Cycles On
Number of effect cycles for the LED to be on
5
Refresh Write
Cycles Off
Number of effect cycles for the LED to be off
5
Refresh Write
Starting Cycle
Normal
Refresh Write
Effect
Constant
Refresh Write
Group
All
Refresh Write
Green Lead
Intensity
Brightness of the LED
0 64 128 192
126
Refresh Write
Cycles On
Number of effect cycles for the LED to be on
5
Refresh Write
Cycles Off
Number of effect cycles for the LED to be off
5
Refresh Write
Starting Cycle
Normal
Refresh Write
Effect
Constant
Refresh Write
Group
All
Refresh Write
Blue Lead
Intensity
Brightness of the LED
0 64 128 192
50
Refresh Write
Cycles On
Number of effect cycles for the LED to be on
5
Refresh Write
Cycles Off
Number of effect cycles for the LED to be off
5
Refresh Write
Starting Cycle
Normal
Refresh Write
Effect
Constant
Refresh Write
Group
All
Refresh Write
Sensor/Turnout creation
Refresh All Save Changes Backup... Restore... More...

MY EXPERIENCE

- Get a working hardware base
- Start with an example and learn to manipulate the CDI
- Once you have the CDI, work on function and event handling
- Set up a bench test with JMRI LCCPro to iterate and test

FUTURE ADDITIONS

- Developing my own Raspberry Pi Pico LCC node to control my turntable. Will use a separate node to control the roundhouse doors and lights.
- Migrate layout accessory lighting to LCC control so that I can tie to a fastclock for operations.
- Add more block detection on industrial spurs.
- Add grade crossings.
- May change command station and boosters to ones with integrated LCC.
- May change reversing loop control to one that is short-free.



RECOMMENDATIONS

- Understand the end game

  decide where you want to be when complete

- LCC is a great solution for turnout control, detection, and signaling. It can be extended to other peripheral uses.

- If you are unhappy with what you have or want LCC
  - Get an LCC starter set
  - Read book, manuals, blogs, sites, and watch some videos
  - Set it up on your workbench and get familiar with how to make it do what you want
  - Deploy to the layout

INFORMATION

- Developer User Group:

  https://groups.io/g/openlcb

- LCC User Group:

  https://groups.io/g/layoutcommandcontrol

  To Join: openlcb+subscribe@groups.iolayoutcommandcontrol+subscribe@groups.io

- Useful Links:

  http://openlcb.org or http://openlcb.com

  https://www.nmra.org/lcc

GETTING UP TO SPEED ON LCC

- Read “Layout Command Control 2025” by Dana Zimmerli. Some of this book is more detailed and technical than a user needs to know for implementation but familiarity of these concepts could be helpful and it won’t hurt (much) to learn it.
- Join the openlcb@groups.io mailing list, which emphasizes LCC development efforts and layoutcommandcontrol@groups.io, which is more user-oriented.
- Look at some of the LCC tutorials found on the Great Northwestern Railway website and video https://www.youtube.com/watch?v=Au4lcuyfx4g
- Introduction to LCC by Robin Peel: https://www.youtube.com/watch?v=jjaNdu0cgaE
- NMRAx clinic “LCC A Better Way to Switch Points” https://www.youtube.com/watch?v=ZsnLhOQSNzs.
- Search for the latest tutorials and videos.
- Look at The OpenLCB Group – Layout Command Control (LCC).
- Decide what you want to do / accomplish with LCC.
- Buy the LCC Starter Kit from RR-CirKits, Inc. , SPROG, or others and perhaps a node or two that align with your goals. Read the manual(s) http://www.rr-cirkits.com/manuals/
- Install jmri A Java Model Railroad Interface if you haven’t already and join jmriusers@groups.io.
- The various groups.io message boards are great sources of support.